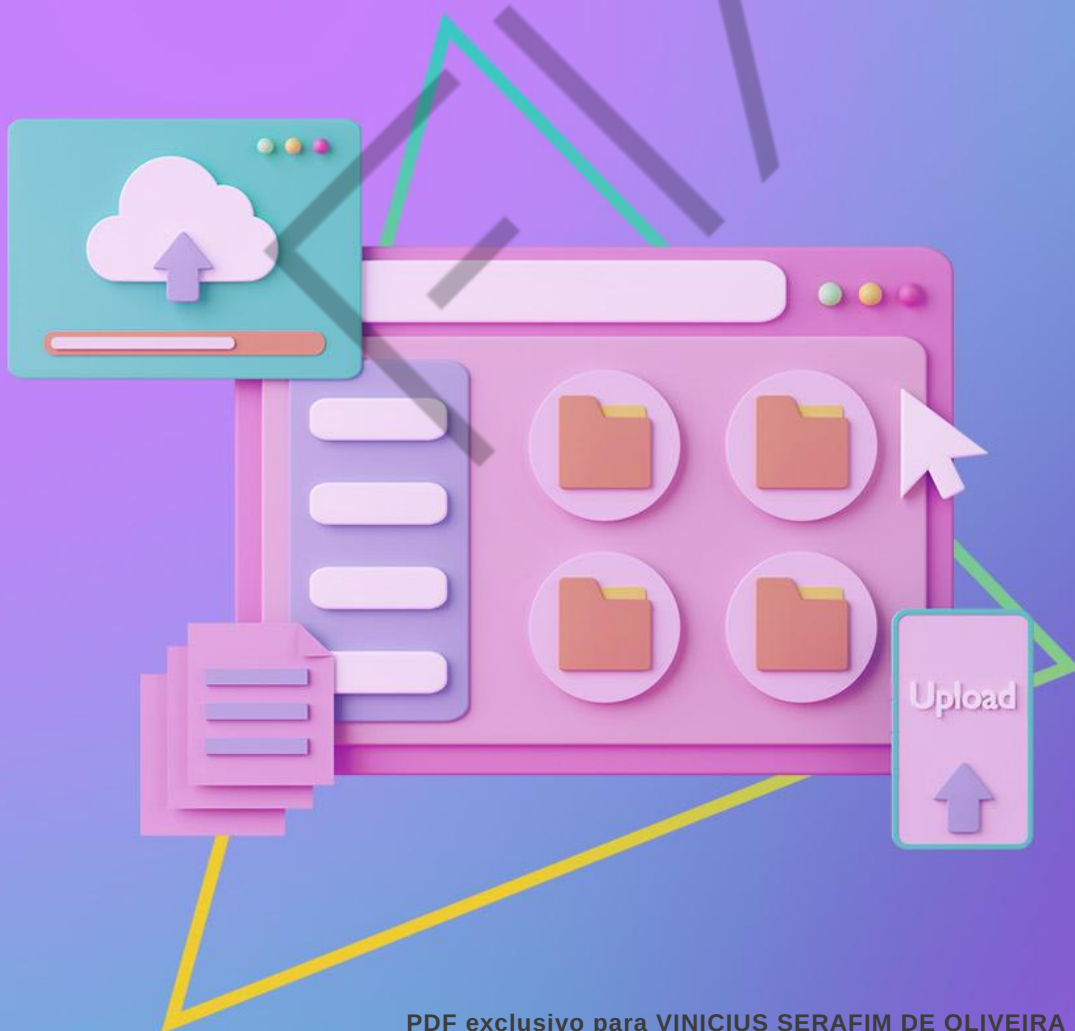


PARA O ALTO E AVANTE

PERÍCIA FORENSE POST-MORTEM II



6

LISTA DE FIGURAS

Figura 1 – Metadados no <i>Linux</i>	8
Figura 2 – Representação das regiões do disco	13
Figura 3 – Exemplo de particionamento	15
Figura 4 - Mapeamento do MBR	16
Figura 5 – Verificação da assinatura do disco	17
Figura 6 – Detalhes da partição primária 1	17
Figura 7 – Estrutura da GPT	19
Figura 8 – Estrutura dos <i>i-nodes</i>	20
Figura 9 – Verificando os <i>i-nodes</i> dos arquivos	21
Figura 10 – Inicialização do <i>Kali Linux</i> em modo forense	21
Figura 11 – Identificando os dispositivos de armazenamento	23
Figura 12 – Verificação da partição de persistência	23
Figura 13 – Ajustes iniciais	24
Figura 14 – Identificação do objeto questionado	25
Figura 15 – Criação da imagem forense	26
Figura 16 – ID do objeto questionado	27
Figura 17 – Informações adicionais do objeto questionado	28
Figura 18 – Formulário de Cadeia de Custódia	29
Figura 19 – Explorando a estrutura do objeto questionado	30
Figura 20 – Listando a imagem	31
Figura 21 – Listando diretórios e arquivos deletados	33
Figura 22 – Recuperação de arquivos deletados	34
Figura 23 – Montagem da imagem	35
Figura 24 – Conteúdo do <i>/mnt</i>	36

LISTA DE QUADROS

Quadro 1 – Diretório acpi	8
Quadro 2 – Sistemas operacionais e <i>file systems</i>	9
Quadro 3 – Conteúdo da partição	32

EMANIP

LISTA DE TABELAS

Tabela 1 – Tamanhos de arquivos e partições ext3.....	12
Tabela 2 – Estrutura do MBR.....	16

EMANIP

SUMÁRIO

PERÍCIA FORENSE POST-MORTEM – II	6
1 INTRODUÇÃO	6
2 PRINCIPAIS ASPECTOS TÉCNICOS	7
2.1 Sistemas de arquivos (<i>filesystems</i>)	7
2.2 Características básicas dos <i>file systems</i>	9
3 PARTICIONAMENTO MBR E GPT	13
3.1 Características básicas do MBR	15
3.2 Características básicas da GPT	18
3.3 Os <i>i-nodes</i>	20
4 HANDS-ON: ANÁLISE POST-MORTEM DE UM PEN DRIVE	21
4.1 Identificando os dispositivos de armazenamento	22
4.2 Ajustes iniciais do ambiente	24
4.3 Criando a imagem forense do objeto questionado	24
4.4 Preenchendo a cadeia de custódia	27
4.5 Analisando a imagem forense	29
4.5.1 Verificando a estrutura do disco	29
4.5.2 Listando o conteúdo da partição	30
4.5.3 Recuperando arquivos deletados	34
4.5.4 Montagem da imagem	35
CONCLUSÃO	36
REFERÊNCIAS	39

PERÍCIA FORENSE POST-MORTEM – II

1 INTRODUÇÃO

Vamos, agora, encerrar o assunto de perícia *post-mortem*. Este conteúdo é muito importante para que possa se desenvolver nesta área com chave de ouro. Por isso, muito foco e atenção!

Conforme já discutido, a perícia forense *post-mortem* remete à perícia das mídias e dispositivos de armazenamento, a partir dos quais a informação eletronicamente armazenada (*electronically stored information* – ESI) poderá ser recuperada, tendo-se como referências concretas e mais comuns *pen drives*, HDs (inclusive externos), CDs e DVDs.

Naturalmente, como em todas as modalidades de perícia forense, a atenção às disposições legais e o alinhamento com modelos de referência reconhecidos e consagrados, como o NIST SP800-86, deverão ser ostensivamente observados. Claro que tais aspectos deverão estar associados ao sólido conhecimento técnico por parte do perito/analista forense, a fim de que os resultados obtidos ao final da investigação sejam satisfatórios.

Há que se atentar ainda às particularidades de cada objeto questionado. Ou seja, a análise de um dispositivo de armazenamento preparado para uso com o sistema operacional Windows apresentará especificidades distintas desse mesmo dispositivo, que foi preparado para armazenamento das informações residentes em sistema baseado em Linux.

De certa forma, o sistema de arquivos aplicado ao dispositivo ou mídia de armazenamento é a parte do sistema operacional com a qual os usuários mais interagem, muitas vezes, sem se darem conta disto. Os sistemas operacionais, geralmente, são capazes de trabalhar com mais de um sistema de arquivos, sendo que a escolha depende de diferentes fatores, como a capacidade de armazenamento do dispositivo e grau de segurança desejada, dentre outros.

Vale mais uma vez reforçar que a recuperação de evidências em procedimentos forenses, no contexto aqui abordado, especificamente arquivos, não tem como finalidade o uso desses para produção, mas sim, a recuperação total ou parcial de seu conteúdo, para uso como evidência em demandas judiciais ou em alinhamento com os objetivos deste estudo, para uso em resposta a incidentes.

Segundo o NIST SP 800-86 - Guia para integração de técnicas forenses em resposta ao incidente (KENT *et al.*, 2006), a motivação mais comumente associada à execução de análises forenses vem da necessidade de melhor entendimento acerca de algum evento de interesse, o que poderá ser alcançado por meio da descoberta e análise dos fatos relacionados a ele.

2 PRINCIPAIS ASPECTOS TÉCNICOS

Muitos aspectos técnicos diferentes deverão ser levados em consideração durante a análise forense de um objeto questionado. Nesta seção, são apresentados alguns dentre os mais relevantes.

2.1 Sistemas de arquivos (*filesystems*)

De maneira geral, pode-se definir o sistema de arquivos como a camada de *software* responsável pelo acesso aos dados armazenados em mídias e dispositivos de armazenamento. Geralmente, tais dados apresentam-se na forma de arquivos, classificados hierarquicamente em subdiretórios (pastas), cabendo ao *file system* garantir a sua integridade e gerenciamento (criação, modificação e exclusão). Cabe acrescentar, ainda, que os *file systems* são responsáveis também pelo armazenamento dos metadados associados aos arquivos, os quais consistem, descrevendo-se de forma sucinta, de dados que descrevem outros dados. A Figura “Metadados no *Linux*” ilustra os metadados associados à listagem do conteúdo do diretório “Evidencias”.

```
user1@fiapon:~/Evidencias$ ls -l /etc
```

total 1056

drwxr-xr-x	3	root	root	4096	Mar	13	2017	acpi
-rw-r--r--	1	root	root	2981	Mar	13	2017	adduser.conf
-rw-r--r--	1	root	root	44	Fev	25	15:14	adjtime
-rw-r--r--	1	root	root	197	Mar	13	2017	aliases
drwxr-xr-x	2	root	root	12288	Mar	13	2017	alternatives
-rw-r--r--	1	root	root	401	Dez	28	2014	anacrontab
drwxr-xr-x	3	root	root	4096	Mar	13	2017	apache2
drwxr-xr-x	3	root	root	4096	Mar	13	2017	apm
drwxr-xr-x	3	root	root	4096	Mar	13	2017	apparmor.d
drwxr-xr-x	6	root	root	4096	Mar	13	2017	apt

Figura 1 – Metadados no *Linux*
Fonte: Elaborado pelo autor (2019)

No destaque vermelho da Figura “Metadados no *Linux*” estão contidos os metadados os arquivos cujos nomes são especificados no destaque azul da mesma figura.

A fim de melhor entender o descrito por esses metadados, tome-se como exemplo o diretório **acpi**, destacado com maior detalhamento no Quadro “Diretório **acpi**”:

```
user1@fiapon:~/Evidencias$ ls -l --full-time /etc
total 1056
drwxr-xr-x 3 root root 4096 2017-03-13 19:03:16.760235000
-0300 acpi
!---várias linhas suprimidas
```

Quadro 1 – Diretório **acpi**
Fonte: Elaborado pelo autor (2019)

O primeiro campo da saída do comando *ls* (Quadro “Diretório **acpi**”) indica o tipo de arquivo, no caso um diretório, representado pela letra “d”. Os três campos subsequentes, em que se observa “rwx r-x r-x” representam as permissões atribuídas, respectivamente, ao usuário (dono do arquivo), grupo primário (do usuário) e outros.

O próximo campo, em que se observa o número 3, indica a quantidade de *hard links* que referenciam o diretório **acpi**. O tamanho do diretório é de 4096 *bytes*, e o *timestamp* indicado no campo a seguir representa a data de criação ou de última modificação do diretório **acpi**. O último campo da listagem em referência indica o nome do arquivo (diretório).

Vale lembrar que diretórios também são um tipo de arquivo os quais, entretanto, não guardam os conteúdos dos arquivos neles contidos, mas apenas seus nomes e respectivos *i-nodes*. Diretórios podem conter qualquer tipo de arquivo, inclusive, outros diretórios.

Os sistemas operacionais são capazes de suportar diferentes *file systems*, a exemplo do exposto no Quadro “Sistemas operacionais e *file systems*”.

Sistema operacional	Exemplos de <i>file systems</i>
Unix / Linux	ext2, ext3, ext4, ReiserFS, JFS, XFS
Android	F2FS, ext4
Windows	Fat12, Fat16, Fat32, NTFS
iOS	HFS, HFS+

Quadro 2 – Sistemas operacionais e *file systems*
Fonte: Adaptado de Lange (2019)

2.2 Características básicas dos *file systems*

Na sequência, é oferecida uma visão geral de alguns dos *file systems* mais amplamente utilizados:

FAT (*File Allocation Table*):

Suportado por grande parte dos sistemas operacionais, recebeu este nome em alusão ao fato de que a principal estrutura do *file system* recai sobre uma tabela de alocação de arquivos. Sua estrutura simples não oferece nenhum tipo de proteção aos dados, entretanto, confere boa performance a pequenos volumes que venham a utilizá-lo. A principal diferença entre a FAT12, FAT16 e FAT32 vem da quantidade de bits utilizados para endereçamento das informações.

O sistema FAT armazena valores numéricos em *bytes* (8 *bits*) ou *words* (dois *bytes*), sendo que, nesse último caso, o armazenamento se dá no modelo **little endian**, ou seja, no primeiro *byte* armazenam-se os *bits* menos significativos, enquanto no segundo *byte* armazenam-se os *bits* mais significativos.

Isso significa que se, por exemplo, considerarmos que um *byte* armazene o valor 0x15 e que o *byte* seguinte armazene o valor 0x74, a *word* formada por eles será 0x7415. O que quer dizer que o valor contido nessa *word* é 29717, e não 5492 (Lange, 2016). A Figura “Estrutura de um volume FAT” ilustra como um *file system* FAT organiza um volume.

Partition Boot Sector	FAT1	FAT2 (duplicate)	Root folder	Other folders and all files.
-----------------------	------	------------------	-------------	------------------------------

O campo *Partiton Boot Sector* (PBS) contém informações que o sistema de arquivos usa para acessar o volume. Em computadores x86, o MBR usa o PBS na partição do sistema para carregar os arquivos do *kernel* do sistema operacional. A *root folder* contém uma entrada para cada arquivo e pasta na raiz. A única diferença entre a pasta raiz e as outras pastas é que a pasta raiz está em um local especificado no disco e tem um tamanho fixo (512 entradas para um disco rígido, o número de entradas em um disquete depende do tamanho do disco).

NTFS (*New Technology File System*):

O NTFS é um *file system* consideravelmente mais seguro que o FAT, permitindo o uso de políticas de segurança e gerenciamento. Aceita volumes de até 2 TB, nos quais o tamanho do arquivo é limitado apenas pelo tamanho do volume, sendo ainda esses volumes menos vulneráveis a fragmentos que o FAT. Embora o NTFS apresente performance geral superior (ainda que mais lento que o FAT32) e se recupere de erros mais facilmente, poderá se corromper com maior facilidade, caso utilizado em mídias removíveis. Partições MBR no NTFS têm identificador 0x07, e GPT EBD0A0A2-B9E5-4433-87C0-68B6B72699C7.

Cabe ressaltar que, como as tabelas de partição em discos MBR limitam o tamanho de partição a até 2 TB, volumes dinâmicos serão necessários à criação de volumes NTFS com tamanho superior a esse limite. A Figura “Estrutura de um volume NTFS” ilustra como um *file system* NTFS organiza um volume.

NTFS Boot Sector	Master File Table	File System Data	Master File Table Copy
------------------	-------------------	------------------	------------------------

O *NTFS Boot Sector* contém o bloco de parâmetros do BIOS que armazena informações sobre o *layout* do volume e as estruturas do sistema de arquivos, bem como o código de inicialização que carrega o sistema operacional. A *Master File Table* (MFT) contém informações necessárias à recuperação de arquivos da partição NTFS, tais como seus atributos; o *File System Data* armazena os dados não contidos na MFT; e, finalmente a *Master File Table Copy* contém os dados necessários à recuperação do *file system* em caso de problemas com a MFT original.

c) Ext3 (*Third Extended File System*):

O ext3 é um sistema de arquivos de *journaling* bastante conhecido do *Linux*, sendo essa (o *journaling*) sua principal diferença em relação ao ext2. Um sistema de arquivos de *journaling* registra as mudanças feitas no *file system* durante operações de gravação no disco a fim de recuperá-lo rapidamente, caso venha a ser corrompido em decorrência de falhas no sistema ou de falta de energia.

O *journaling* realizado pelo sistema de arquivos pode ser configurado para diferentes níveis de registro, dependendo das necessidades e requisitos de desempenho. Os três níveis de *journaling* disponíveis no ext3 são:

- **Journal:** no qual os dados e metadados dos arquivos são registrados no *journal* antes de serem escritos no *file system* principal. Dessa forma, a segurança dos dados é consideravelmente incrementada, entretanto, com penalização do desempenho, uma vez que serão escritos por duas vezes no disco.
- **Writeback:** os metadados dos arquivos são escritos no *journal*, enquanto o conteúdo do arquivo é escrito diretamente no disco. Essa abordagem oferece desempenho bastante superior ao modo *journal*, entretanto, preserva apenas a integridade da estrutura do *file system*. Os dados (conteúdo do arquivo) poderão ser corrompidos, por exemplo, em caso de travamento do sistema após o registro dos metadados, mas antes dos blocos de dados serem gravados. Esse problema poderá ser resolvido por meio da adoção do modo *ordered*.
- **Ordered:** nesse modo, os metadados serão registrados no *journal* após a gravação dos dados no disco, assim sendo, a consistência dos dados e do *file system* poderão ser garantidos após uma recuperação.

O ext3 tem como identificadores da partição os valores 0x83 (MBR) e EBD0A0A2-B9E5-4433-87C0-68B6B72699C7 (GPT).

É importante salientar que, diferentemente do ext2, o ext3 zera os ponteiros de blocos nos *i-nodes* de arquivos apagados a fim de simplificar o acesso de leitura e escrita quando o *journal* está sendo utilizado após uma montagem "suja". Entretanto, isso dificulta consideravelmente a recuperação dos arquivos. O único recurso do

usuário é fazer um "*grep*" no disco rígido em busca por dados que sejam sinalizadores de início/fim de arquivo.

O ext3 suporta partições com até 32 TB de dados e arquivos com tamanho de até 2TB, sendo tais valores variáveis de acordo com a Tabela "Tamanhos de arquivos e partições ext3".

Tamanho do bloco	Tamanho máximo do arquivo	Tamanho máximo do <i>filesystem</i>
1KB	16GB	2TB
2KB	256GB	8TB
4KB	2TB	16TB
8KB	2TB	32TB

Tabela 1 – Tamanhos de arquivos e partições ext3
Fonte: Adaptado de Lange (2019)

d) Ext4 (*Fourth Extended Filesystem*):

Sucessor do ext3, o ext4 extrapola as limitações desse em relação ao tamanho máximo para arquivos e para o *file system*. Utilizando endereços de 48 *bits*, o ext4 suporta partições com até 1 EB (*exabyte*) e arquivos com até 16 TB. Além dessas características, outros pontos relevantes que diferenciam o ext4 do ext3 são:

- **Alocação tardia**, ou seja, os dados não são escritos imediatamente no disco, o que melhora o desempenho e diminui a fragmentação dos arquivos, já que o *file system* pode esperar e, assim, determinar a melhor forma de gravação dos arquivos no disco. Tal prática, entretanto, poderá aumentar a possibilidade de corrompimento de arquivos.
- **Checagem do *journal* (*checksumming*)**: o ext4 realiza esta checagem de modo a garantir uma restauração mais rápida e resistente a falhas. Característica especialmente interessante a grandes sistemas.
- **Não apresenta limite à criação de pastas**: o ext4 não apresenta este limite, diferentemente do ext3, que tem limite de 32.000 pastas.
- **Desfragmentação on-line**: ou seja, o ext4 desfragmenta enquanto os arquivos estão sendo alocados. Por sua vez, o ext3 não apresenta nenhuma ferramenta para tal finalidade.

- **Criação de uma ferramenta *UNDELETE***: talvez seja este o maior destaque do ext4 em relação ao ext3, impedindo que determinados arquivos e pastas venham a ser apagados.
- **Checagem rápida do sistema de arquivos**: ou seja, a ferramenta *FSCK* poderá ignorar áreas não alocadas ao verificar partições ext4, diminuindo consideravelmente o tempo total de verificação da partição.

3 PARTICIONAMENTO MBR E GPT

Quando o IBM PC foi lançado pela primeira vez, ele veio com um sistema operacional de disco simples chamado MS-DOS, fornecido pela *Microsoft*. A IBM tomou a decisão de que um PC poderia ter até quatro sistemas operacionais instalados ao mesmo tempo. Cada sistema operacional teria sua própria parte do disco, chamada de partição. Por esse motivo, o primeiro setor de cada disco em um PC, chamado de MBR (*Master Boot Record*), contém uma tabela com quatro entradas, **cada uma dando os locais inicial e final das quatro partições**.

A ordem das quatro entradas no MBR não precisa corresponder à ordem real das partições do disco, e pode haver lacunas entre as partições, antes da primeira e depois da última. Alguns podem afirmar que no pior dos casos, um disco pode ter até nove regiões: quatro partições e cinco intervalos, conforme ilustrado pela Figura “Representação das regiões do disco”.

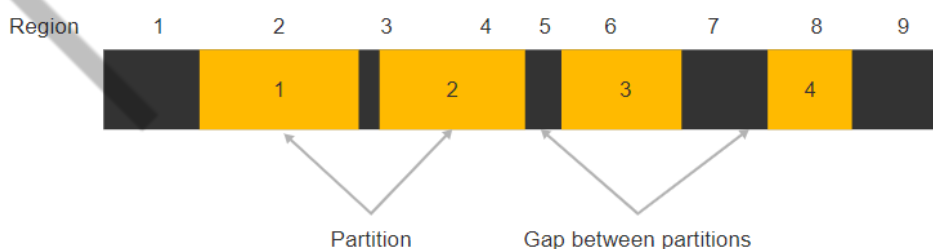


Figura 2 – Representação das regiões do disco
Fonte: Minix3.org (2019)

De maneira simplificada, portanto, particionar um disco significa dividi-lo em múltiplas unidades lógicas denominadas partições, como se diferentes discos coexistissem dentro de um único disco físico. Também é possível que um único sistema operacional utilize múltiplas partições. Por exemplo, um usuário pode decidir

particionar o disco com duas partições: uma para o sistema operacional e outra para seus dados. Esse arranjo facilita o *backup* dos dados do usuário, permitindo, ainda, a instalação de uma nova versão do sistema operacional existente sem qualquer prejuízo a esses dados.

Não tardou para que IBM e Microsoft decidissem que quatro partições não seriam suficientes. Ao invés de simplesmente ampliar a tabela de partições no MBR, tornando-a incompatível com versões anteriores do MS-DOS, foi decidido que até três partições poderiam ser marcadas como *EXTENDED* (estendida), significando que elas conteriam uma ou mais subpartições denominadas unidades lógicas.

Uma partição que não seja estendida é denominada partição primária. Juntas, as partições primária e estendida compõem as quatro partições de nível superior. Há que se ressaltar que quase todos os *softwares* de particionamento de disco exibem as partições primárias e estendidas, bem como as unidades lógicas, de maneira que possam parecer todas iguais. Isso, definitivamente, não corresponde à verdade e acaba se tornando bastante confuso, pois cada unidade lógica é uma subpartição de alguma partição estendida envolvente. Além disso, alguns *softwares* de gerenciamento de partições as exibem na ordem em que aparecem no MBR, e não na ordem de seus endereços de disco.

O MS-DOS e o Windows são normalmente configurados com uma partição primária e, opcionalmente, uma partição estendida. Na maioria dos casos, a partição primária é chamada C:.

Uma partição estendida, se presente, conterá as unidades lógicas D:, E:, e assim sucessivamente, dependendo de quantas unidades lógicas foram criadas nessa partição.

A vantagem em se alocar as unidades D:, E:, e demais, dentro de uma partição estendida, em vez de alocar cada uma em sua própria partição primária, é que ela deixa os *slots* da tabela de partição do MBR disponíveis para outros sistemas operacionais. A Figura “Exemplo de particionamento” representa um disco hipotético com uma partição primária (C:) e uma partição estendida contendo duas unidades lógicas (D: e E:).

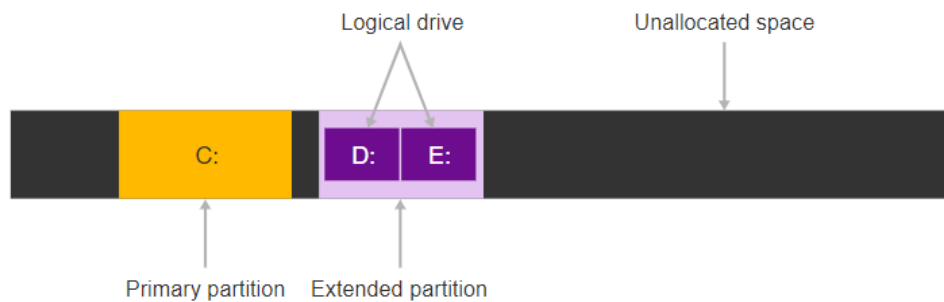


Figura 3 – Exemplo de particionamento
Fonte: Minix3.org (2019)

É importante destacar que o conceito de letras de unidade, como C:, é um conceito do *MS-DOS*, posteriormente passado também ao *Windows*. Nenhuma variante do *UNIX*, como *BSD*, *Linux* ou *MINIX 3*, usa letras de unidade.

Todos os sistemas *UNIX* simplesmente lidam com as quatro partições de nível superior no MBR.

3.1 Características básicas do MBR

O disco pode ser dividido em até quatro partições primárias, sendo possível, ainda, a divisão do disco em até três partições primárias e uma partição estendida. Uma partição estendida pode dividir-se em múltiplas partições lógicas.

O MBR usa 32 *bits* para armazenar informações das partições estando, portanto, o tamanho máximo de cada partição limitado a 2TB.

Toda informação das partições é guardada em um único local. A Figura “Estrutura do MBR” ilustra a estrutura do *master boot record*.

Endereço		Descrição		Tamanho em Bytes
Hex	Dec			
0000	0	Código de arranque do SO		440 (max. 446)
01B8	440	Assinatura opcional		4
01BC	444	normalmente nulo ; 0x0000		2
01BE	446	Tabela de partições primarias (Quatro entradas de 16 bytes (IBM Partition Table scheme))		64
01FE	510	55h	MBR signature; 0x55AA	2
01FF	511	AAh		
Tamanho total do MBR : 440 + 4 + 2 + 64 + 2 =				512

Tabela 2 – Estrutura do MBR
Fonte: Wikipedia (2019)

Com base nas informações cedidas pela Figura “Estrutura do MBR”, é possível mapear facilmente as entidades nela descritas, conforme exemplificado pela Figura “Mapeamento do MBR”.

00000000	eb	63	90	10	8e	d0	bc	00	b0	b8	00	00	8e	d8	8e	d0	
00000010	fb	be	00	7c	bf	00	06	b9	00	02	f3	a4	ea	21	06	00	
00000020	00	be	be	07	38	04	75	0b	83	d6	10	81	fe	fe	07	75	
00000030	f3	eb	16	b4	02	b0	01	bb	00	7c	b2	80	8a	74	01	8b	
00000040	4c	02	cd	13	ea	00	7c	00	00	eb	fe	00	00	00	00	00	
00000050	00	00	00	00	00	00	00	00	00	00	00	80	01	00	00	00	
00000060	00	00	00	00	ff	fa	90	90	f6	c2	80	74	05	f6	c2	70	
00000070	74	02	b2	80	ea	79	7c	00	00	31	d0	8e	d8	8e	d0	bc	
00000080	00	20	fb	a0	04	7c	3c	ff	74	02	88	c2	52	bb	17	04	
00000090	f6	07	03	74	06	be	88	7d	e8	17	01	be	05	7c	b4	41	
000000a0	bb	aa	55	cd	13	5a	52	72	3d	81	fb	55	aa	75	37	83	
000000b0	e1	01	74	32	31	d0	89	44	04	40	88	44	ff	89	44	02	
000000c0	c7	04	10	00	06	8b	1e	5c	7c	06	89	5c	08	66	8b	1e	
000000d0	60	7c	66	89	5c	0c	d7	44	06	00	70	b4	42	cd	13	72	
000000e0	05	bb	00	70	eb	76	b4	08	cd	13	73	0d	5a	84	d2	0f	
000000f0	83	d0	00	be	93	7d	e9	82	00	06	0f	b6	d6	88	64	ff	
00000100	40	66	89	44	04	0f	b6	d1	c1	e2	02	88	e8	88	14	40	
00000110	89	44	08	0f	b6	d2	d0	e8	02	06	89	04	66	a1	60	7c	
00000120	66	09	d0	75	4e	66	a1	5c	7c	06	31	d2	66	f7	34	88	
00000130	d1	31	d2	66	f7	74	04	3b	44	08	7d	37	fe	c1	88	c5	
00000140	30	d0	c1	e8	03	08	c1	88	d0	5a	88	d6	bb	00	70	8e	
00000150	c3	31	db	b8	01	02	cd	13	72	1e	8c	c3	60	1e	b9	00	
00000160	01	8e	db	31	f6	bf	00	80	8e	d6	fc	f3	a5	1f	61	ff	
00000170	26	5a	7c	be	8e	7d	eb	03	be	9d	7d	e8	34	00	be	a2	
00000180	7d	e8	2e	00	cd	18	eb	fe	47	52	55	42	20	00	47	65	
00000190	6f	6d	00	48	61	72	64	20	44	69	73	6b	00	52	65	61	
000001a0	64	00	20	45	72	72	6f	72	0d	0a	00	bb	01	00	b4	0e	
000001b0	cd	10	ac	3c	00	75	f4	c3	39	f6	5f	d6	00	00	80	20	
000001c0	21	00	83	56	38	1e	00	08	00	00	00	68	07	00	00	56	
000001d0	39	1e	83	ae	aa	04	00	70	07	00	00	38	77	00	00	ae	
000001e0	ab	04	83	ca	93	f7	00	a8	7e	00	00	98	3b	00	00	ea	
000001f0	b2	f7	05	fe	ff	ff	fe	47	ba	00	02	b0	45	00	55	aa	
00000200																	

Figura 4 - Mapeamento do MBR
Fonte: Elaborado pelo autor (2019)

Com base na estrutura do MBR anteriormente ilustrada, observa-se que o destaque na cor marrom da Figura “Mapeamento do MBR” indica a assinatura do disco (39 f6 5f d6, lembrando-se que sua leitura deverá ser invertida, a fim de obter-se o valor correto, portanto: d6 5f f6 39).

Embora esse valor seja opcional, diferentes sistemas operacionais – incluindo *Linux* (*kernel* 2.6 e posteriores) e *Windows* (*Windows* NT e posteriores) – o utilizam como um identificador único do disco.

A Figura “Verificação da assinatura do disco” ilustra o uso da ferramenta *fdisk* para verificação da estrutura e assinatura do disco (destaque superior).

```

root@fiapon:/home/user1/Evidencias# fdisk /dev/sda

Bem-vindo ao fdisk (util-linux 2.25.2).
As alterações permanecerão apenas na memória, até que você decida gravá-las.
Tenha cuidado antes de usar o comando de gravação.

Comando (m para ajuda) : p
Disco /dev/sda: 8 GiB, 8589934592 bytes, 16777216 setores
Tamanho: setor de 1 * 512 = 512 bytes
Tamanho de setor (lógico/físico) : 512 bytes / 512 bytes
Tamanho E/S (mínimo/ótimo): 512 bytes / 512 bytes
Tipo de rótulo do disco : dos
Identificador do disco : 0xd65 ff639

Device      Boot      Start      End      Sectors  Size  Id  Type
/dev/sda1   *          2048     487423   485376   237M  83  Linux
/dev/sda2             487424   8300543   7813120   3,7G  83  Linux
/dev/sda3             8300544  12206079   3905536   1,9G  83  Linux
/dev/sda4          12208126  16775167   4567042   2,2G   5  Extended
/dev/sda5          12208128  15136767   2928640   1,4G  83  Linux
/dev/sda6          15138816  16775167   1636352   799M  82  Linux Swap / Solaris

```

Figura 5 – Verificação da assinatura do disco

Fonte: Próprio autor (2019)

Ainda com base na Figura “Mapeamento do MBR”, os *offsets* destacados nas cores azul (0x01BE a 0x01CD), verde (0x01CE a 0x01DC), amarelo (0x01DE a 0x01ED) e laranja (0x01EE a 0x01FD) correspondem, respectivamente, às partições primárias 1 a 4. Em alguns sistemas operacionais, o *offset* em vermelho (0x01BC a 0x01BD) pode assumir dois valores: 0x0000 indicando que o disco não apresenta proteção contra gravação, e 0x5A5A indicando que o disco está protegido contra gravação.

000001b0	cd	10	ac	3c	00	75	f4	c3	39	f6	5f	d6	00	00	80	20
000001c0	21	00	83	56	38	1e	00	08	00	00	00	68	07	00	00	56

Figura 6 – Detalhes da partição primária 1

Fonte: Elaborado pelo autor (2019)

No destaque da Figura “Detalhes da partição primária 1”, o valor 0x80 observado no *offset* 0x01BE indica ser essa a partição de *boot* do sistema. O valor 0x00 observado em *offsets* análogos nas demais partições indica que estas estão inativas.

Ainda na faixa destinada à partição 1, o valor 0x83 armazenado no *offset* 0x04 dessa (Figura “Detalhes da partição primária 1”) indica o tipo da partição, no caso, uma partição de sistema de arquivos nativo do *Linux*. Tal fato pode ser confirmado pelo destaque inferior da Figura “Verificação da assinatura do disco”.

Os parâmetros analisados, bem como os demais aqui omitidos são detalhados na documentação do MBR.

3.2 Características básicas da GPT

A GPT (*GUID Partition Table*) é um sistema de particionamento que utiliza o conceito de Identificador Único Global (*Globally Unique Identifier* – GUID). O termo GUID indica um identificador gerado aleatoriamente, com grande probabilidade de ser exclusivo, dado o grande universo de combinações possíveis (2^{128}) para geração das chaves. O GUID é composto por sequências hexadecimais com 8,4,4,4 e 12 dígitos separados por hífen, por exemplo, a19d880f-05fc-4d3b-a006-743f0f84911e o qual corresponde ao identificador de uma partição RAID do *Linux*.

A Figura “Estrutura da GPT” ilustra o *layout* da GPT.

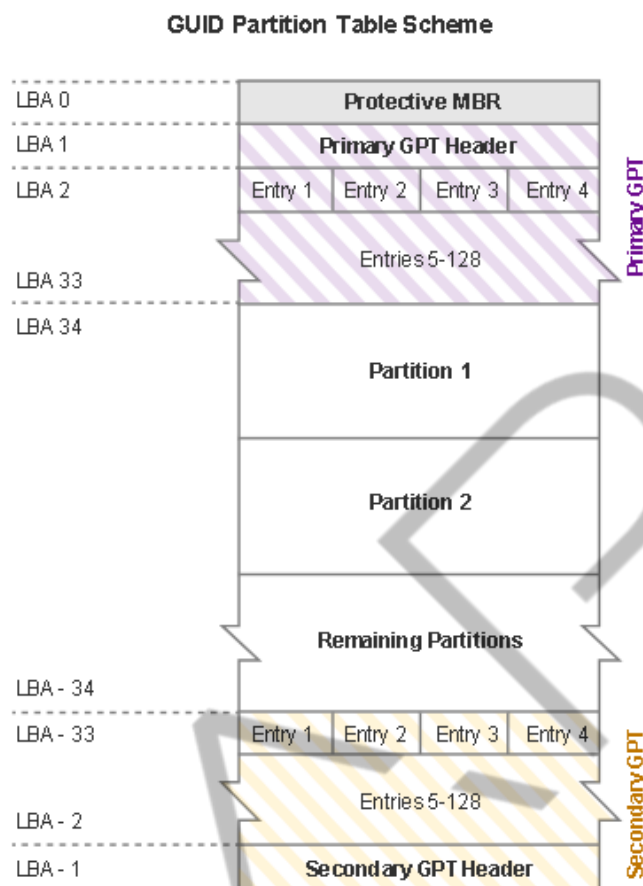


Figura 7 – Estrutura da GPT
Fonte: Wikipedia (2019)

Por motivos de compatibilidade, a GPT grava um MBR de proteção (*Protective MBR*) no primeiro setor do disco (LBA0), de forma que aplicativos ou sistemas operacionais incapazes de ler discos GPT sejam impedidos de apagar informações necessárias ao funcionamento do sistema. No campo em que o MBR armazenaria o valor correspondente ao tipo de partição, a GPT armazena o valor 0xEE, indicando que o esquema GPT encontra-se em vigor.

No segundo setor do disco (LBA1), é gravado o cabeçalho GPT, o qual determina os blocos que poderão ser usados no disco. Dentre outras vantagens da GPT sobre o MBR, destaca-se aqui a capacidade dessa em suportar partições maiores que 2TB e a possibilidade de criação de até 128 partições primárias.

3.3 Os *i-nodes*

O conteúdo dos arquivos é armazenado em blocos: grupos contíguos de setores do disco, também referenciados como *clusters* em *file systems* NTFS e FAT. Já os metadados dos arquivos são armazenados em estruturas denominadas *i-nodes*.

A Figura “Estrutura dos *i-nodes*” ilustra a estrutura dos *i-nodes* nos *file systems* ext2/ext3.

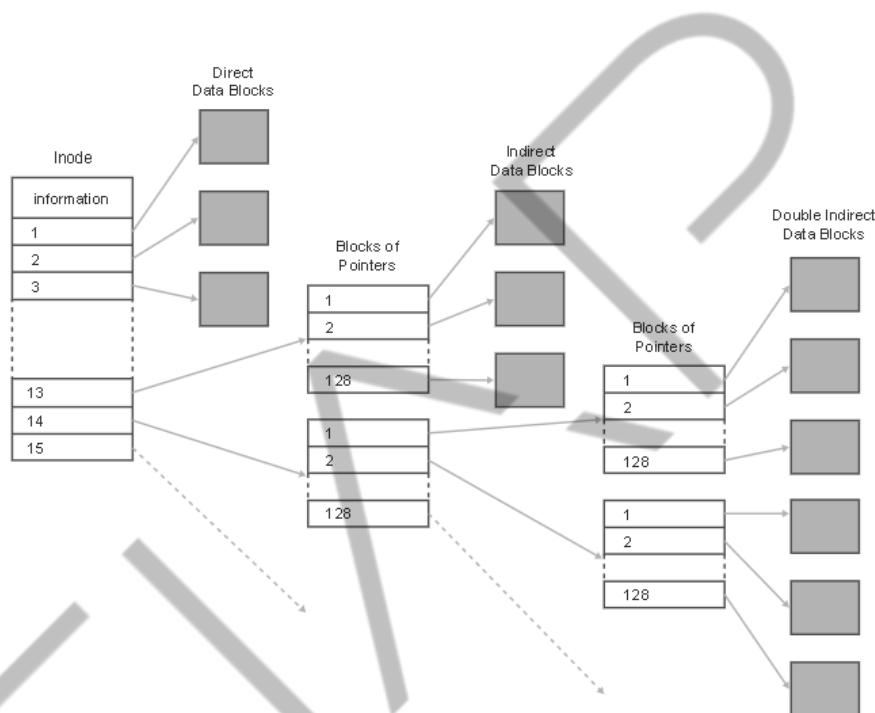


Figura 8 – Estrutura dos *i-nodes*
Fonte: Wikipedia (2019)

Da Figura “Estrutura dos *i-nodes*” é possível observar que existem 15 apontadores, sendo que entre o 1 e o 12 são apontados blocos diretos. O 13 aponta para um bloco indireto, o 14 aponta para um bloco indireto duplo e o 15 para um bloco indireto triplo (extrapolando-se o efetivamente indicado pela figura).

A Figura “Verificando os *i-node* dos arquivos” demonstra como exibir os *i-nodes* dos arquivos no *Linux*.

```
user1@fiapon:~/Evidencias$ ls -lhi
total 1,1M
1173 -rw-rw-r-- 1 user1 3,8K Fev 21 01:15 arquivo1.txt
972 -rw-r--r-- 1 user1 13K Fev 19 00:19 fiapon
1519 -rw-r--r-- 1 user1 474K Out 27 2017 Hiding_finding_data.pdf
3038 -rw-r--r-- 1 user1 564K Fev 26 20:31 LINUX.pdf
1284 -rw-r--r-- 1 root 512 Fev 22 01:48 mbr.raw
user1@fiapon:~/Evidencias$
```

Figura 9 – Verificando os *i-nodes* dos arquivos
Fonte: Wikipedia (2019)

A sintaxe utilizada para o comando *ls* na Figura “Verificando os *i-node* dos arquivos” inclui os argumentos *-lhi* produzindo, respectivamente: uma saída longa (detalhada, com metadados), com os tamanhos dos arquivos representados em “formato humano” adequado (no caso, KB), mais os *i-nodes* dos arquivos listados.

4 HANDS-ON: ANÁLISE POST-MORTEM DE UM PEN DRIVE

A fim de demonstrar de forma prática a aplicação dos conceitos abordados, será executada a análise de um *pen drive* com ajuda do *Sleuthkit*, integrante do *Kali Linux*. É importante observar que o próprio *Kali Linux* deverá ser instalado em um *pen drive* a fim de possibilitar a inicialização da estação de trabalho do investigador em **modo forense** (Figura “Inicialização do *Kali Linux* em modo forense”).



Figura 10 – Inicialização do *Kali Linux* em modo forense
Fonte: Top-Password (2019)

Esse modo foi inicialmente introduzido no *BackTrack Linux* e apresenta duas características especialmente adequadas ao uso forense:

O disco rígido interno do *host* inicializado nesse modo é totalmente preservado, não sendo acessado pelo sistema operacional em nenhum momento. Assim sendo, caso o próprio objeto questionado venha a ser inicializado dessa maneira, as evidências contidas em seu HD serão preservadas, incluindo-se a memória virtual (partição *swap* no caso de sistemas *Linux*, e o arquivo *pagefile.sys* em sistemas *Windows*), dentre outros elementos.

O recurso *auto-mount* de mídias (dispositivos) removíveis é desativado, também com a finalidade de preservação de seu conteúdo.

O procedimento para criação de um *pen drive* bootável do *Kali Linux* está disponível em: <<https://www.kali.org/docs/usb/live-usb-install-with-linux/>>. Incluir uma partição para persistência de dados para armazenamento das evidências.

Considere-se, em um cenário hipotético, um *pen drive* enviado pelo departamento jurídico ao analista forense da empresa para verificação de seu conteúdo. Admitindo-se que o analista forense utilizará seu *notebook* de trabalho na condução da investigação, o procedimento será iniciado bootando-se o referido *host* com o *Kali Linux* em modo forense, conforme anteriormente descrito.

4.1 Identificando os dispositivos de armazenamento

Como primeira providência no ambiente do *Kali Linux*, serão identificados os dispositivos de armazenamento (HDs e *pen drives* dentre outros) conectados à estação de trabalho do analista forense. Para tal, pode-se utilizar o comando *lsblk*, conforme ilustrado na Figura “Identificando os dispositivos de armazenamento”.

```

root@kali:~# lsblk
NAME        MAJ:MIN     RM  SIZE  RO  TYPE  MOUNTPOINT
loop0       7:0         0   2.5G  1   loop  /lib/live/mount/rootfs/filesystem.squashfs
sda         8:0         0  931.5G  0   disk
├─sda1      8:1         0   500M  0   part
├─sda2      8:2         0  390.1G  0   part
├─sda3      8:3         0  279.4G  0   part
├─sda4      8:4         0     1K  0   part
├─sda5      8:5         0  251.9G  0   part
├─sda6      8:6         0   9.3G  0   part
└─sda7      8:7         0   238M  0   part
sdb         8:16        1   29.1G  0   disk
├─sdb1      8:17        1   2.7G  0   part  /lib/live/mount/medium
└─sdb2      8:18        1  26.5G  0   part  /media/root/PERSISTENCE
sr0        11:0         1  1024M  0   rom

```

Disco rígido do sistema

Partições do disco

Pendrive Kali

Partição de boot

Partição da persistência

Figura 11 – Identificando os dispositivos de armazenamento

Fonte: Elaborado pelo autor (2019)

Da Figura “Identificando os dispositivos de armazenamento”, é possível observar que existem dois dispositivos de armazenamento atualmente conectados ao *host*: o seu disco rígido, identificado como **sda**, e o *pen drive* do *Kali Linux*, identificado como **sdb**. Observa-se também a partição **sdb2**, criada para persistência de dados.

ATENÇÃO: Vale reforçar que, ao referir-se simplesmente a **sda**, faz-se referência ao primeiro dispositivo de armazenamento do sistema (no caso, o HD da estação forense), como um todo. Ao referir-se a **sdb2**, faz-se referência à segunda partição do dispositivo de armazenamento **sdb** – no caso, a partição para persistência de dados criada no *pen drive* do *Kali Linux*.

Identificada a partição destinada à persistência de dados, deve ser verificado, então, se esta já se encontra disponível (montada). Para isso, deve-se utilizar o comando *df*, conforme ilustrado pela Figura “Verificação da partição de persistência”.

```

root@kali:~# df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  3.9G   0    3.9G   0% /dev
tmpfs           tmpfs     789M   9.6M  779M   2% /run
/dev/sdb1       iso9660   2.7G   2.7M   0    100% /lib/live/mount/medium
/dev/loop0      squashfs  2.5G   2.5M   0    100% /lib/live/mount/rootfs/filesystem.squa
shfs
tmpfs           tmpfs     3.9G   0    3.9G   0% /lib/live/mount/overlay
overlay         overlay   3.9G   9.5M  3.9G   1% /
tmpfs           tmpfs     3.9G   0    3.9G   0% /dev/shm
tmpfs           tmpfs     5.0M   4.0K  5.0M   1% /run/lock
tmpfs           tmpfs     3.9G   0    3.9G   0% /sys/fs/cgroup
tmpfs           tmpfs     3.9G   8.0K  3.9G   1% /tmp
tmpfs           tmpfs     789M   32K  789M   1% /run/user/0
/dev/sdb2       vfat      27G    5.0M  27G    1% /media/root/PERSISTENCE

```

Ponto de montagem da partição

Figura 12 – Verificação da partição de persistência

Fonte: Elaborado pelo autor (2019)

Na Figura “Verificação da partição de persistência” observa-se que a referida partição está montada em /media/root/PERSISTENCE.

4.2 Ajustes iniciais do ambiente

Disponível a partição para persistência de dados, o relógio do sistema deverá ser verificado e corretamente ajustado, caso necessário. Isso poderá ser feito diretamente pela *interface* gráfica ou por meio do comando *date* em um terminal texto. Sugere-se que os registros referentes à data/hora do sistema e ao ambiente também sejam devidamente preservados, conforme ilustrado na Figura “Ajustes iniciais”.

```
root@kali:/media/root/PERSISTENCE/Forense# date | tee date.txt
Wed Mar  6 16:30:10 UTC 2019
root@kali:/media/root/PERSISTENCE/Forense# cat /etc/os-release | tee os-release.txt
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
ID=kali
VERSION="2018.2"
VERSION_ID="2018.2"
ID_LIKE=debian
ANSI_COLOR="1;31"
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://bugs.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
root@kali:/media/root/PERSISTENCE/Forense# █
```

Figura 13 – Ajustes iniciais
Fonte: Elaborado pelo autor (2019)

Na Figura “Ajustes iniciais” pode-se observar que os comandos executados, respectivamente: ***date*** e ***cat /etc/os-release***, tiveram suas saídas redirecionadas pelo *pipe* (|) para arquivos texto com os mesmos nomes dos comandos, gravados na partição de persistência de dados (/media/root/PERSISTENCE/Forense/<nome_arq>.txt), preservando, assim, informações referentes à data e hora de início dos procedimentos (arquivo *date.txt*) e detalhes dos ambientes utilizados (arquivo *os-release*).

4.3 Criando a imagem forense do objeto questionado

Finalizados os procedimentos iniciais, pode-se iniciar a criação da imagem forense do objeto questionado. Para isso, deve ser conectado à estação forense.

Como, entretanto, o recurso de *auto-mount* do sistema operacional está desabilitado, a fim de preservar o objeto questionado, esse terá de ser identificado manualmente.

Não há necessidade de montagem do objeto questionado para criação de sua imagem forense.

Depois de conectado à estação forense, a ferramenta *lsblk* poderá ser novamente utilizada para identificação do objeto questionado (marcado em vermelho na Figura “Identificação do objeto questionado”), a exemplo do já executado.

```
root@kali:~# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
loop0       7:0    0   2.5G  1  loop /lib/live/mount/rootfs/filesystem.squashfs
sda         8:0    0 931.5G  0  disk
├─sda1      8:1    0   500M  0  part
├─sda2      8:2    0 390.1G  0  part
├─sda3      8:3    0 279.4G  0  part
├─sda4      8:4    0    1K    0  part
├─sda5      8:5    0 251.9G  0  part
├─sda6      8:6    0    9.3G  0  part
└─sda7      8:7    0   238M  0  part
sdb         8:16   1   29.1G  0  disk
├─sdb1      8:17   1    2.7G  0  part /lib/live/mount/medium
└─sdb2      8:18   1   26.5G  0  part /media/root/PERSISTENCE
sdc         8:32   1   281M  0  disk
└─sdc1      8:33   1   480M  0  part
sr0         11:0    1 1024M  0  rom
```

Identificação do objeto questionado

Figura 14 – Identificação do objeto questionado
Fonte: Elaborado pelo autor (2019)

Do destaque na Figura “Identificação do objeto questionado” é possível observar que o objeto questionado é o dispositivo **sdc**, podendo-se agora, então, proceder à criação da respectiva imagem forense, conforme ilustrado pela Figura “Criação da imagem forense”.

```

root@kali:~# cd /media/root/PERSISTENCE/Forense/
root@kali:/media/root/PERSISTENCE/Forense # dcfldd if=/dev/sdc of=dump.raw bs=512 hash=
md5,sha1 hashlog=hashes.dump [1]
985088 blocks (481Mb) written.
985088+0 records in
985088+0 records out
root@kali:/media/root/PERSISTENCE/Forense #
root@kali:/media/root/PERSISTENCE/Forense # md5sum /dev/sdc > md5sum.sdc [2]
root@kali:/media/root/PERSISTENCE/Forense # sha1sum /dev/sdc > sha1sum.sdc
root@kali:/media/root/PERSISTENCE/Forense #
root@kali:/media/root/PERSISTENCE/Forense # cat md5sum.sdc
eac9f42fac79445229fcaabf4f66ecb /dev/sdc
root@kali:/media/root/PERSISTENCE/Forense # grep eac9f42fac79445229fcaabf4f66ecb hashe
s.dump
Total (md5) : eac9f42fac79445229fcaabf4f66ecb
root@kali:/media/root/PERSISTENCE/Forense # cat sha1sum.sdc
1a42601045897b0c9f7fc551448d0aa38e37b818 /dev/sdc
root@kali:/media/root/PERSISTENCE/Forense # grep 1a42601045897b0c9f7fc551448d0aa38e37b8
18 hashes.dump
Total (sha1): 1a42601045897b0c9f7fc551448d0aa38e37b818
root@kali:/media/root/PERSISTENCE/Forense # █

```

[3] Comparação entre os hashes da imagem e do objeto questionado

Figura 15 – Criação da imagem forense
Fonte: Elaborado pelo autor (2019)

No destaque 1 da Figura “Criação da imagem forense”, é possível verificar o uso da ferramenta *dcfldd* para criação da imagem forense do objeto questionado, uma ferramenta mais nova e completa que a tradicional *dd*. Com sintaxes bastante semelhantes, também nessa nova ferramenta, o arquivo de origem da imagem forense é designado pela opção *if* (*if=/dev/sdc*), bem como o arquivo de destino da imagem é designado pela opção *of*, o qual será gravado no diretório corrente, sob o nome de *dump.raw* (*of=dump.raw*). Ainda como na ferramenta *dd*, a opção *bs* da *dcfldd* especifica o tamanho dos blocos a serem copiados – no caso 512 bytes (*bs=512*).

As diferenças entre essas ferramentas surgem da (I) possibilidade oferecida pela *dcfldd* de obter o *hash on the fly* da imagem produzida por meio da opção *hash*. No caso, solicita-se a extração de dois *hashes* diferentes (md5 e sha1), os quais (II) serão armazenados em arquivo denominado *hashes.dump* (*hash=md5, sha1 hashlog=hashes.dump*).

O destaque 2 da mesma figura indica a extração e armazenamento dos *hashes* md5 e sha1 – agora do próprio objeto questionado, os quais, em sequência, são comparados aos *hashes* extraídos da imagem forense (destaque 3).

A imagem produzida só será considerada íntegra e admissível caso seus *hashes* coincidam com os *hashes* do próprio objeto questionado.

Atestada a integridade da imagem forense, uma cópia de trabalho desta deverá ser produzida, tendo também sua integridade devidamente verificada, analogamente

ao procedimento anteriormente descrito. Mesmo a cópia de trabalho deverá ter dois *hashes* extraídos (no caso, md5 e sha1) e comparados aos *hashes* da imagem forense original. A cópia de trabalho será admissível somente se ambos coincidirem.

É importante destacar ainda que, embora o NIST considere tanto o md5 quanto sha1 algoritmos inseguros para aplicações voltadas à geração de assinaturas digitais, de *timestamps* e outras aplicações que requeiram resistência a colisões, no contexto aqui empregado, tais algoritmos ainda podem ser aceitos.

4.4 Preenchendo a cadeia de custódia

Finalizadas as tarefas das fases de preservação e coleta da evidência digital, o objeto questionado poderá ser desconectado da estação de trabalho do analista, devendo-se, agora, preencher a CADEIA de CUSTÓDIA. Para isso, o ID do objeto questionado deverá ser devidamente identificado, conforme ilustrado na Figura “ID do objeto questionado”.

```
root@kali:/media/root/PERSISTENCE/Forense# lsusb | tee lsusb.txt
Bus 002 Device 004: ID 0489:e00d Foxconn / Hon Hai Broadcom Bluetooth 2.1 Device
Bus 002 Device 003: ID 5986:0364 Acer, Inc
Bus 002 Device 005: ID 0930:651e Toshiba Corp. TravelDrive 2C
Bus 002 Device 002: ID 8087:0024 Intel Corp. Integrated Rate Matching Hub
Bus 002 Device 001: ID 1d6d:0002 Linux Foundation 2.0 root hub
Bus 001 Device 004: ID 0bda:0139 Realtek Semiconductor Corp. RTS5139 Card Reader Controller
Bus 001 Device 003: ID 03f0:6c40 Hewlett-Packard
Bus 001 Device 002: ID 8087:0024 Intel Corp. Integrated Rate Matching Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
root@kali:/media/root/PERSISTENCE/Forense# █
```

Figura 16 – ID do objeto questionado

Fonte: Elaborado pelo autor (2019)

Por meio da ferramenta *lsusb*, foi possível a obtenção do ID do objeto questionado: 0930:651e, conforme ilustrado pelo destaque na Figura “ID do objeto questionado”.

Basta, agora, a repetição do mesmo comando, entretanto, de forma mais específica, a fim de se obter informações detalhadas a respeito do objeto questionado (Figura “Informações adicionais do objeto questionado”).

Vale ressaltar que, mais uma vez, as saídas dos comandos executados deverão ser devidamente gravadas em arquivos, a fim de comprovar os procedimentos realizados.

```
root@kali:/media/root/PERSISTENCE/Forensics# lsusb -d 0930:651e -v | tee lsusb2.txt
```

Bus 002 Device 005: ID 0930:651eToshiba Corp. TravelDrive 2C

Device Descriptor:

bLength	18	
bDescriptorType	1	
bcdUSB	2.00	
bDeviceClass	0	(Defined at Interface level)
bDeviceSubClass	0	
bDeviceProtocol	0	
bMacPacketSize0	64	
idVendor	0x0930	Toshiba Corp.
idProduct	0x651e	TravelDrive 2C
bcdDevice	1.00	
iManufacturer	1	Kingston
iProduct	2	DataTraveler 2.0
iSerial	3	085113517230B957

Informações adicionais necessárias ao preenchimento da CADEIA de CUSTÓDIA

Figura 17 – Informações adicionais do objeto questionado

Fonte: Elaborado pelo autor (2019)

O comando executado na Figura “Informações adicionais do objeto questionado” (*lsusb*) fornece informações detalhadas (-v) do dispositivo especificado (-d 0930:651e), trazendo o identificador do fabricante (*iManufacturer*), o identificador do produto (*iProduct*) e o número de série (*iSerial*) do dispositivo, dentre outras informações necessárias ao preenchimento da CADEIA de CUSTÓDIA, conforme ilustrado na Figura “Formulário de Cadeia de Custódia”.

Formulário de Cadeia de Custódia				
Caso nº: 0001-19		Página: 01/01		
Detalhes do equipamento / mídia eletrônica				
Item:	Descrição:			
01	Unidade de armazenamento removível USB (pendrive), 512MB			
Fabricante:	Modelo:	Núm. de série:		
Kingston	DataTraveler 2.0	08511357230B957		
Detalhes da imagem dos dados				
Hora:	Data:	Criada por:	Nome:	Tipo:
16:56	06/03/2019	Arruda, O.D.	dump.raw	física
MD5:		SHA1:		
eac9f42fac79445229fcaabf4f66ecb		1a42601045897b0c9f7fc551448c0aa38e37b818		
Cadeia de Custódia				
Origem		Destino		ID
Data:	Departamento jurídico	Data:	CSIRT	01
06/03/2019		06/03/2019		
Hora:		Hora:		
16:25		17:22		
Data:		Data:		
Hora:		Hora:		

Figura 18 – Formulário de Cadeia de Custódia
Fonte: Elaborado pelo autor (2019)

4.5 Analisando a imagem forense

Produzida e certificada a imagem forense de trabalho, o analista poderá iniciar sua análise, buscando, por exemplo, recuperar arquivos íntegros nela contidos, recuperar arquivos apagados e, ainda, verificar arquivos que eventualmente tenham suas extensões alteradas (como uma forma primária de ocultação do arquivo) dentre outras possibilidades.

4.5.1 Verificando a estrutura do disco

Deve-se iniciar com a verificação da estrutura do objeto questionado, a partir da cópia de trabalho da imagem forense, conforme indicado na Figura “Explorando a estrutura do objeto questionado”.

```

root@kali:/media/root/PERSISTENCE/Forens# mmls dump.raw | tee mmls.txt
DOS Partition Table
Offset Sector: 0
Units are in 512-bytes sectors

    Slot      Start      End      Length    Description
000: Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001: -----  0000000000  0000002047  0000002048  Unallocated
002: 000:000  0000002048  0000005087  0000983040  Win95 FAT32 (0x0b)
root@kali:/media/root/PERSISTENCE/Forens# █

```

Figura 19 – Explorando a estrutura do objeto questionado
Fonte: Elaborado pelo autor (2019)

A Figura “Explorando a estrutura do objeto questionado” ilustra o uso da ferramenta *mmls* do *Sleuthkit*, a qual exibe o *layout* utilizado quando do particionamento do volume (objeto questionado). Embora essa ferramenta possibilite o uso de vários argumentos, o único obrigatório dentre esses no contexto abordado é o nome da imagem a ser analisada (*mmls dump.raw*).

ATENÇÃO: Os valores apontados pela saída da ferramenta referem-se a **setores do disco**.

Assim sendo, pode-se concluir que o objeto questionado tem (I) uma única partição, (II) formatada em FAT32 – identificada pelo valor 0x0b, (III) essa partição se inicia no setor 2048 do disco, (IV) se estendendo até o seu final (destaque na Figura “Explorando a estrutura do objeto questionado”). Com base em tais informações, observa-se que para acesso à referida partição, as ferramentas deverão ser configuradas para utilização de um deslocamento (*offset*) de 2048 (setores).

4.5.2 Listando o conteúdo da partição

Certamente, uma das tarefas mais importantes do analista forense recai sobre a listagem do conteúdo do objeto questionado. Entretanto, há que se ter sempre em mente, estar trabalhando com um tipo especial de arquivo: uma imagem forense, a qual representa fielmente a estrutura e o conteúdo de um dispositivo de armazenamento (objeto questionado). Logo, se for empregada uma ferramenta tradicional – a *ls -lh*, por exemplo –, será obtida a listagem longa do próprio arquivo de imagem e não a listagem de seu conteúdo, conforme ilustrado na Figura “Listando a imagem”.

```
root@kali:/media/root/PERSISTENCE/Forense# ls -lh dump.raw
-rw-r--r-- 1 root root 481M Mar  6 16:56 dump.raw
root@kali:/media/root/PERSISTENCE/Forense# █
```

Figura 20 – Listando a imagem
Fonte: Elaborado pelo autor (2019)

Do exposto, observa-se que para ter acesso ao conteúdo da imagem, ferramentas adequadas deverão utilizadas.

A ferramenta *fls* do *Sleuthkit* permite a listagem do conteúdo de uma imagem de disco (arquivos e diretórios).

Acesse
Veja <http://www.sleuthkit.org/sleuthkit/man/fls.html> para maiores informações

Para obter a listagem do conteúdo da imagem analisada, basta utilizar a seguinte linha de comando:

```
root@kali:/media/root/PERSISTENCE/Forense# fls -o 2048 dump.raw | tee fls.txt
```

Essa linha terá como saída a listagem do conteúdo da partição da imagem *dump.raw*, iniciada no setor 2048 do disco, sendo ainda esse conteúdo preservado em um arquivo chamado *fls.txt*.

A listagem (reduzida) do conteúdo da imagem pode ser vista no Quadro “Conteúdo da partição”.

```

[1] r/r 3: bob (Volume Label Entry)
[2] r/r 7: partially_frozen_pond_197983.jpg
[3] r/r 11: partially_frozen_pond_197985.jpg
[4] r/r 51: homero-iliada.zip
[5] r/r 54: homero-odisseia.zip
[6] r/r 57: maquiavel-o-principe.zip
[7] r/r 61: dante-alighieri-a-divina-comedia.zip
[8] r/r 65: O-Misterio-de-Sittaford.pdf
[9] r/r 68: Ramones - Surfin_Bird.mp4
[10] r/r 82: hping2.pdf
[11] r/r * 83: _NALIS~1.PDF
[12] r/r 85: refP12.pdf
[13] r/r 87: refp13.pdf
[14] r/r 90: tendencias_graficos.xls
[15] r/r 92: custos.pdf
[16] r/r 95: anonymous-1573691.jpg
[17] r/r 98: concorrentes.txt~
[18] r/r * 99: _LOCKA~1.XLS
[19] r/r 103: indicadores_financeiros2.xls
[20] r/r 106: hping-examples
[21] r/r * 107: _NDICA~2.XLS
[21] r/r 109: agregados.xls
[22] r/r * 110: _LOCKE~1.XLS
[23] r/r 114: planejamento_estrategico.xls
[24] r/r 118: como-criar-o-seu-negocio-online.pdf
[25] r/r 121: foca_linux2.pdf
[26] r/r 124: foca_linux1.pdf
[27] r/r * 127: cartao_esposa.gif
[28] r/r 129: notas.txt
[29] r/r 131: notas2.txt
!
!---muitas linhas suprimidas
!
[30] d/d 152: System Volume Information
[31] r/r 156: LM_102_08_09_02_coluna_alexandre.pdf
[32] r/r 159: misc_tools_sheet_v1.pdf
[33] r/r 162: bhdc08-slides-fyodor.pdf
[34] r/r 164: nmap.pdf
[35] d/d 248: .Trash-1000
[36] r/r 253: Segurança com Software Livre - IPTABLES I.docx
[37] r/r 258: Segurança com Software Livre - IPTABLES I.pdf
[38] r/r 267: IP Spoofing.pdf
[39] r/r 271: Wireshark_Display_Filters.pdf
[40] r/r * 273: concl116.ods
[41] r/r * 274: _OUTPU~1
[42] v/v 15720707: $MBR
[43] v/v 15720708: $FAT1
[44] v/v 15720709: $FAT2
[45] V/V 15720710: $OrphanFiles

```

Quadro 3 – Conteúdo da partição
Fonte: Elaborado pelo autor (2019)

Da linha [1] do Quadro “Conteúdo da partição”, observa-se que o nome dado ao volume é Bob, fornecendo, portanto, indício de quem pode ser o dono ou custodiante do dispositivo.

A fim de conhecer o formato padrão da saída do *fls*, tome-se como exemplo a linha [2] do Quadro “Conteúdo da partição”:

```
r/r 7:  partially_frozen_pond_197983.jpg
```

O parâmetro *r/r* indica o tipo de arquivo, no caso, um arquivo regular (por exemplo, um arquivo texto ou uma imagem). O *fls* pode identificar diversos tipos de arquivos, dentre eles: (d) diretórios, (b) dispositivos de bloco, (l) *link simbólico* e (h) *socket*, dentre outros. O número 7 indica o número do *i-node* do arquivo *partially_frozen_pond_197983.jpg*. Na linha [11] do mesmo quadro, o asterisco que segue os caracteres *r/r*, geralmente, indica que o referido arquivo foi deletado.

Conforme ilustrado na Figura “Listando diretórios e arquivos deletados”, o comando *fls -o 2048 dump.raw -d* pode ser utilizado para listar apenas arquivos deletados, enquanto o comando *fls -o 2048 dump.raw -D* pode ser utilizado para listar apenas diretórios.

```
root@kali:/media/root/PERSISTENCE/Forense# fls -o 2048 dump.raw -d
r/r * 83 :      _NALIS~1.PDF
r/r * 99 :      _LOCKA~1.XLS
r/r * 107 :     _NDICA~2.XLS
r/r * 110 :     _LOCKE~1.XLS
r/r * 127 :     cartao_esposa.gif
r/r * 189 :     Cartilha do Arduino (Muito bom!).pdf
r/r * 273 :     conc1116.ods
r/r * 274 :     _OUTPU~1
root@kali:/media/root/PERSISTENCE/Forense# fls -o 2048 dump.raw -D
d/d 152:      System Volume Information
d/d 248:      .Trash-1000
V/V 15720710: $OrphanFiles
root@kali:/media/root/PERSISTENCE/Forense# █
```

Figura 21 – Listando diretórios e arquivos deletados
Fonte: Elaborado pelo autor (2019)

Vale destacar que essas são apenas duas das diversas capacidades do *fls*, portanto, a leitura de sua documentação poderá tornar o trabalho do analista consideravelmente mais produtivo.

4.5.3 Recuperando arquivos deletados

Na linha [27] do Quadro “Conteúdo da partição” observa-se a saída:

```
r/r * 127:    cartao_esposa.gif
```

Do anteriormente exposto, pode-se concluir ser *cartao_esposa.gif* um arquivo regular (r/r), deletado (*), o qual ainda pode ser acessado por meio de seu *i-node* (127). A recuperação do arquivo poderá ser feita pela ferramenta *icat*, a qual extrai o conteúdo de um arquivo referenciado com base no respectivo *i-node*.

A Figura “Recuperação de arquivos deletados” exhibe a utilização dessa ferramenta.

```
root@kali :/media/root/PERSISTENCE/Forense# icat -s -o 2048 dump.raw 127 > cartao
_esposa.gif
root@kali :/media/root/PERSISTENCE/Forense# ls -l cartao_esposa.gif
-rw-r--r-- 1 root root 8192 Mar  7 20:12 cartao_esposa.gif
root@kali :/media/root/PERSISTENCE/Forense# file cartao_esposa.gif
cartao_esposa.gif: OpenDocument Spreadsheet
root@kali :/media/root/PERSISTENCE/Forense# █
```

Figura 22 – Recuperação de arquivos deletados
Fonte: Elaborado pelo autor (2019)

Na Figura “Recuperação de arquivos deletados” a ferramenta *icat* recupera o conteúdo do arquivo identificado pelo *i-node* 127 contido na imagem *dump.raw*, redirecionando a saída (>) para um arquivo denominado *cartão_esposa.gif*, conforme listado na Figura “Listando diretórios e arquivos deletados”. Caso tal redirecionamento não houvesse sido feito, o conteúdo do arquivo extraído seria enviado à tela (descriptor de saída padrão do sistema operacional). O argumento *-s* utilizado com a ferramenta faz com que *slack spaces* sejam incluídos na saída.

Embora o arquivo inicialmente aparentasse ser uma imagem, dada sua extensão (.gif), a ferramenta *file* indica que, na verdade, o arquivo é uma planilha de cálculos (*OpenDocument Spreadsheet*). Portanto, talvez a eventual troca da extensão (e possivelmente do nome todo), associada à deleção do arquivo, possa ser indício de uma tentativa de ocultação dess.

Naturalmente, para que todo o conteúdo do objeto questionado possa passar por esse processo de verificação, o analista forense deverá se valer de *scripts* para

que possa trabalhar com eficiência, ou utilizar-se, por exemplo, do *Autopsy* – uma interface gráfica para o *Sleuthkit* e outras ferramentas para investigação digital.

4.5.4 Montagem da imagem

Até o momento, toda a interação com a imagem forense do objeto questionado tem sido feita por meio de ferramentas especializadas. Entretanto, pode ser que em alguma situação específica o analista tenha necessidade de interagir diretamente com a imagem, tratando-a como se fosse o objeto questionado conectado à estação de trabalho do analista forense.

Isso pode ser feito facilmente no *Linux*, por meio do comando *mount*, nativo em qualquer distribuição. Todavia, dois cuidados básicos devem ser tomados: (a) a imagem deverá ser montada com permissões apenas para leitura (*read only*) e (b) o comando *mount* deverá utilizar um *loop device* para poder montar a partição (da imagem) como se fosse um dispositivo de bloco tradicional.

Assim sendo, o comando *mount* precisará de um *offset* preciso a fim de localizar com exatidão a posição da partição a ser montada. Entretanto, diferentemente das ferramentas forenses – feitas para trabalho com imagens, este *offset* não deverá ser calculado em **setores**, mas sim, em **bytes**. Portanto, considerando-se que cada setor contenha 512 *bytes*, o *offset* a ser utilizado corresponderá ao produto de **2048 x 512 = 1.048.576**

A Figura “Montagem da imagem” ilustra o processo de montagem da imagem, e verificação da partição.

```
root@kali:/media/root/PERSISTENCE/Forens# mount -o ro,loop,o ffset=1048576 dump.
raw/mnt
root@kali:/media/root/PERSISTENCE/Forens# df -h T
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
udev	devtmpfs	3.9G	0	3.9G	0%	/dev
tmpfs	tmpfs	789M	9.7M	779M	2%	/run
/dev/sdb1	iso9660	2.7G	2.7G	0	100%	/lib/live/mount/medium
/dev/loop0	squashfs	2.5G	2.7G	0	100%	/lib/live/mount/rootfs/filesystem
.squashfs						
tmpfs	tmpfs	3.9G	0	3.9G	0%	/lib/live/mount/overlay
overlay	overlay	3.9G	24M	3.9G	1%	/
tmpfs	tmpfs	3.9G	0	3.9G	0%	/dev/shm
tmpfs	tmpfs	5.0G	4.0K	5.0M	1%	/run/lock
tmpfs	tmpfs	3.9G	0	3.9G	0%	/sys/fs/cgroup
tmpfs	tmpfs	3.9G	8.0K	3.9G	1%	/tmp
tmpfs	tmpfs	789M	28K	789M	1%	/run/user/0
/dev/sdb2	vfat	27G	487M	26G	2%	/media/root/PERSISTENCE
/dev/loop1	vfat	480M	96M	385M	20%	/mnt

```
root@kali:/media/root/PERSISTENCE/Forens# █
```

Figura 23 – Montagem da imagem

Fonte: Elaborado pelo autor (2019)

Na linha de comando executada na Figura “Montagem da imagem”, as opções (-o) utilizadas com o comando *mount: ro,loop* e *offset*, respectivamente, (a) montam a imagem somente para leitura (*read only*), (b) criam um *loop device* e (c) definem um *offset* de 1048576 *bytes* para que a imagem possa ser montada no diretório */mnt*. Ainda na mesma figura, o destaque inferior confirma o sucesso da montagem.

A partir de agora, o conteúdo da imagem poderá ser acessado como um diretório comum do sistema, conforme demonstrado pela Figura “Conteúdo do /mnt”, em que é possível observar uma listagem parcial do conteúdo da partição (da imagem) montada. Para desmontagem da imagem basta executar o comando:

```
root@kali:/media/root/PERSISTENCE/Forense# umount /mnt
```

```
root@kali:/media/root/PERSISTENCE/Forense# ls -l /mnt
total 493976
-rwxr-xr-x 1 root root 76461 Mar 6 18:23 ajustes_iniciais.png
-rwxr-xr-x 1 root root 9770 Mar 7 21:45 bodyfile.txt
-rwxr-xr-x 1 root root 145396 Mar 6 17:38 criacao_imagem.png
-rwxr-xr-x 1 root root 29 Mar 6 17:38 date.txt
-rwxr-xr-x 1 root root 504365056 Mar 6 17:38 dump.raw
-rwxr-xr-x 1 root root 71138 Mar 7 19:51 fls-dA.png
-rwxr-xr-x 1 root root 95860 Mar 7 19:45 fls-d.png
-rwxr-xr-x 1 root root 2970 Mar 7 18:21 fls.txt
-rwxr-xr-x 1 root root 103 Mar 6 17:38 hashes.dump
-rwxr-xr-x 1 root root 50185 Mar 7 20:17 icat2.png
-rwxr-xr-x 1 root root 75376 Mar 7 20:14 icat.png
-rwxr-xr-x 1 root root 74149 Mar 6 17:38 identificado_dispositivos_armazenamento.png
-rwxr-xr-x 1 root root 113616 Mar 7 14:37 ident_oq2.png
-rwxr-xr-x 1 root root 93978 Mar 7 14:35 ident_oq.png
-rwxr-xr-x 1 root root 85194 Mar 6 17:38 identificacao_objeto_questionado.png
-rwxr-xr-x 1 root root 128681 Mar 7 22:35 loop.png
-rwxr-xr-x 1 root root 681 Mar 6 17:38 lsblk.txt
-rwxr-xr-x 1 root root 35650 Mar 7 17:25 ls-lh.png
-rwxr-xr-x 1 root root 43 Mar 6 17:38 md5sum.sdc
-rwxr-xr-x 1 root root 62205 Mar 7 16:52 mmls.png
```

Figura 24 – Conteúdo do /mnt
Fonte: Elaborado pelo autor (2019)

CONCLUSÃO

Naturalmente, a execução manual dos procedimentos tem como finalidade demonstrar de forma prática os principais conceitos envolvidos no processo da análise forense *post-mortem*.

Em casos reais, sugere-se o uso do *Sleuthkit* por meio do *Autopsy*, presentes na maioria das distribuições especializadas como o *Caine*, *Deft* e o próprio *Kali Linux* dentre outros. O *Autopsy* provê um ambiente simples, porém, completo e adequado à manipulação e armazenamento da evidência digital.

Sugere-se, ainda, uma pesquisa um pouco mais aprofundada também em relação aos grandes *players* do mercado, por exemplo, o *EnCase*, dando-se aqui ainda destaque especial ao IPED – Indexador e Processador de Evidências Digitais, *software* amplamente utilizado na Operação Lava Jato, desenvolvido pelo perito criminal federal e professor Luís Filipe da Cruz Nassif.

Vale destacar que, enquanto o *EnCase* é uma ferramenta proprietária fabricado pela *Guidance Software*, o IPED pode ser livremente instalado e utilizado (*free software*).

Um rápido *overview* de algumas das características do IPED inclui:

Processamento de Imagens Forenses: o IPED é capaz de acessar o conteúdo de dispositivos físicos e imagens forenses DD, 001, E01 e ISO por meio da *suite* forense *The Sleuthkit (TSK)* e *Libewf*, os quais são utilizados para acessar o conteúdo das imagens e decodificação dos sistemas de arquivos. Além disso, por meio do *Sleuthkit*, é realizada, automaticamente, a recuperação simples de arquivos apagados das tabelas dos sistemas de arquivos. Ainda por intermédio do TSK, é possível também o acesso ao espaço não alocado, o qual é indexado e submetido ao *data carving* otimizado pelas tarefas de processamento específicas do IPED.

Categorização: a categorização dos arquivos é realizada principalmente via análise de assinatura pela biblioteca [Apache Tika](#). A biblioteca retorna o *mimeType* do arquivo, o qual é mapeado para uma categoria conforme configurado no arquivo: *conf/CategoriesByTypeConfig.txt*.

Deteção de Arquivos Criptografados: a partir da versão 3.3 é realizada automaticamente deteção de arquivos criptografados dos tipos: pdf, office97 (doc, xls, ppt), office2007 (docx, xlsx, pptx), openoffice (odt, ods, odp), zip, rar, 7z e pst. Os arquivos identificados como cifrados podem ser acessados por um filtro previamente configurado.

Cálculo de múltiplos Hashes: atualmente, o *software* suporta os seguintes algoritmos de *hash* criptográfico: md5, sha-1, sha-256, sha-512, edonkey.

E assim voltaremos à superfície, agora para tratarmos de negócios. Aproveite o tempo que levará para viajar até a superfície e estude um pouco mais a fundo sobre perícia forense computacional. Com certeza valerá a pena.

Em fases posteriores, você ainda verá um pouco mais de perícia computacional, como a análise de tráfego de dados em uma rede, mas, se conseguir, não espere até lá, esse é um conteúdo muito rico e que pode ajudar você em muitas missões e batalhas cibernéticas.

EMANIP

REFERÊNCIAS

CAINE. [s.d.]. Disponível em: <<https://www.caine-live.net/>>. Acesso em: 31 maio 2021.

EDRM. **EDRM Model**. [s.d.]. Disponível em: <<https://www.edrm.net/frameworks-and-standards/edrm-model/>>. Acesso em: 31 maio 2021.

ELEUTÉRIO, P. M. S.; MACHADO, M. P. **Desvendando a computação forense**. 4. Reimpressão. São Paulo: Novatec, 2014.

FARMER, D.; VENEMA, W. **Perícia Forense Computacional** – Teoria e Prática Aplicada. 4. Reimpressão. São Paulo: Pearson Prentice Hall, 2011.

IBM. **Anatomy of Linux journaling file systems**. Disponível em: <<https://www.ibm.com/developerworks/library/l-journaling-file-systems/index.html>>. Acesso em: 31 maio 2021.

KENT, K et al. **Guide to Integrating Forensic Techniques into Incident Response**. Gaithersburg: Information Technology Laboratory National Institute of Standards and Technology, 2006. Disponível em: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf>. Acesso em: 13 jun. 2022.

LANGE, R. **Tratado de computação forense**. Campinas: Millennium, 2016.

LINUXTOPIA. **What is the ext3 filesystem?** Disponível em: <<https://www.linuxtopia.org/HowToGuides/ext3JournalingFilesystem.html>>. Acesso em: 31 maio 2021.

MICROSOFT. **How NTFS works**. Disponível em: <[https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc781134\(v=ws.10\)](https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc781134(v=ws.10))>. Acesso em: 31 maio 2021.

MICROSOFT. **Visão geral dos arquivos FAT, HPFS e NTFS**. Disponível em: <<https://support.microsoft.com/pt-br/help/100108/overview-of-fat-hpfs-and-ntfs-file-systems>>. Acesso em: 31 maio 2021.

MINIX3. **Introduction to partitions**. [s.d.]. Disponível em: <<http://www.minix3.org/doc/partitions.html>>. Acesso em: 31 maio 2021.

NIST. **Hash Functions**. Disponível em: <<https://csrc.nist.gov/Projects/Hash-Functions/NIST-Policy-on-Hash-Functions>>. Acesso em: 31 maio 2021.

NTFS. **FAT file system**. Disponível em: <<http://www.ntfs.com/fat-systems.htm>>. Acesso em: 31 maio 2021.

OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S.S. **Sistemas Operacionais**. 4. ed. Porto Alegre: Bookman e Instituto de Informática de UFRGS, 2010.

RAMOS, M. **Peritos da PF produzem 1,6 mil laudos no âmbito da Lava Jato**. [s.d.]. Disponível em: < [Peritos da PF produzem 1,6 mil laudos no âmbito da Lava Jato - Época \(globo.com\)](#)>. Acesso em: 31 maio 2021.

SANS. **Sift Work Station**. [s.d.]. Disponível em: <<https://digital-forensics.sans.org/community/downloads>>. Acesso em: 31 maio 2021.

SILBERSCHATS, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 9. ed. USA: John Wiley & Sons Inc., 2012.

SLEUTH KIT. [s.d.]. Disponível em: <<https://www.sleuthkit.org/>>. Acesso em: 18 fev. 2019.

VANDERBURG, E. **MAC Times in computer forensics**. 2009. Disponível em: <<https://www.tcdi.com/mac-times-in-computer-forensics/>>. Acesso em: 31 maio 2021.